



PFA · 2025–2026

DevPilot AI

Plateforme agentique de RAG
observable, traçable et explicable

Transformer des documents privés en réponses ancrées, citées et auditable.

RÉALISÉ PAR

Bilal Malih
Oussama Mahi

ENCADRÉ PAR

Pr. Widad ELOUATAOUI



Une soutenance, une démonstration, trois preuves

Notre question : peut-on rendre un système RAG aussi inspectable qu'un service logiciel critique ?

CONTEXTE

1 · COMPRENDRE

Le problème métier

Pourquoi la recherche classique et les assistants généralistes ne suffisent pas pour une connaissance privée.

CONCEPTION

2 · CONSTRUIRE

L'architecture réelle

Ingestion asynchrone, recherche hybride, pipeline spécialisé, sécurité et streaming.

RÉSULTATS

3 · PROUVER

La valeur observable

Démonstration, traces, RAGOps, tests versionnés, limites et perspectives.

Objectif de la présentation : montrer non seulement la réponse, mais aussi les preuves de sa production.

Le problème : la connaissance existe, mais reste difficile à exploiter

Connaissance fragmentée

Spécifications
Documentation d'architecture
Comptes rendus
Procédures internes
Décisions techniques

Recherche limitée

Les mots-clés trouvent des chaînes de caractères, mais comprennent mal l'intention, les synonymes et le contexte.

Réponse opaque

Une réponse plausible sans source, sans trace et sans score de qualité est difficile à vérifier et risquée à utiliser.

Question directrice : comment centraliser, retrouver, générer, mesurer et auditer ?

Notre réponse : une plateforme RAG conçue autour de la preuve

DevPilot AI associe la recherche documentaire à une couche d'exploitation et de gouvernance.

01

ANCRER

Recherche hybride
Contexte privé
Citations [n]

02

MESURER

Fidélité
Pertinence
Risque d'hallucination

03

TRACER

Étapes
Chunks & scores
Latences

04

GOVERNER

JWT & RBAC
Workspaces
Audit

Différenciation : la réponse + les sources + le déroulement + le signal de qualité.

Périmètre livré

FONCTIONNEL

- ✓ Authentification et rôles
- ✓ Workspaces isolés
- ✓ Upload PDF / TXT / Markdown
- ✓ Ingestion asynchrone
- ✓ Recherche sémantique, lexicale et hybride
- ✓ Chat cité, évaluation et historique
- ✓ RAGOps, traces et studio d'exécution

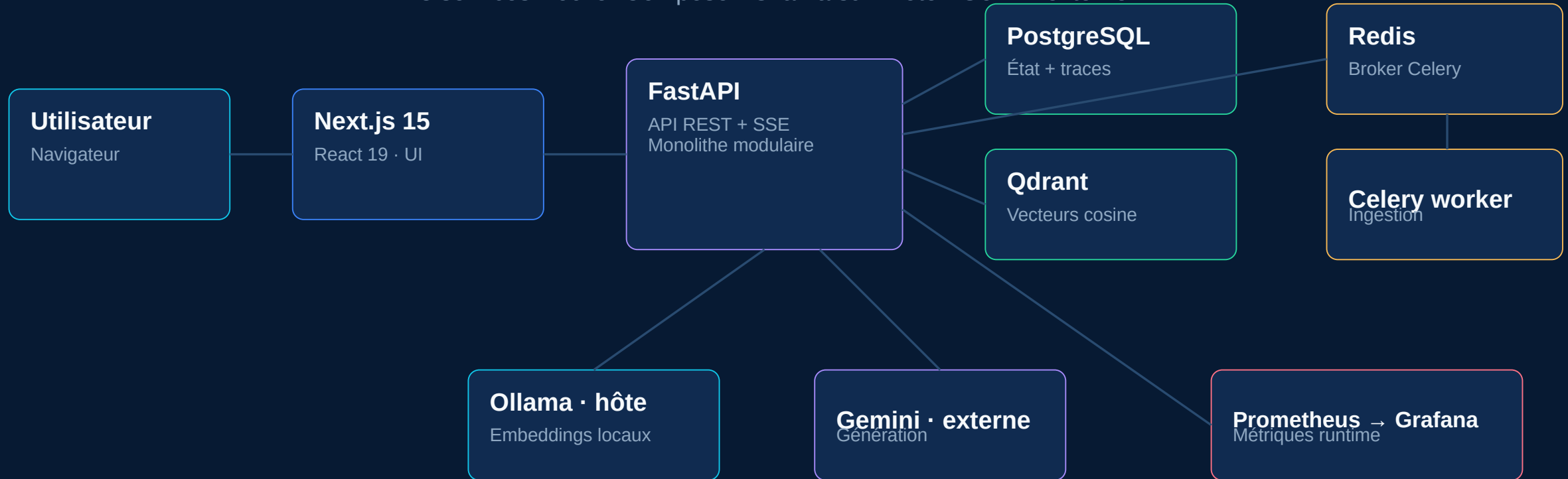
NON FONCTIONNEL

- ✓ Sécurité : JWT, RBAC, filtres workspace
- ✓ Réactivité : API async + SSE
- ✓ Résilience : retries et fallbacks ciblés
- ✓ Maintenabilité : couches + typage + migrations
- ✓ Observabilité : logs, métriques, traces
- ✓ Déploiement : Docker Compose
- ✓ Transparence : limites documentées

Hors périmètre actuel : haute disponibilité, Kubernetes, multi-tenant SaaS et benchmark RAGAS.

Architecture globale : monolithe modulaire + worker asynchrone

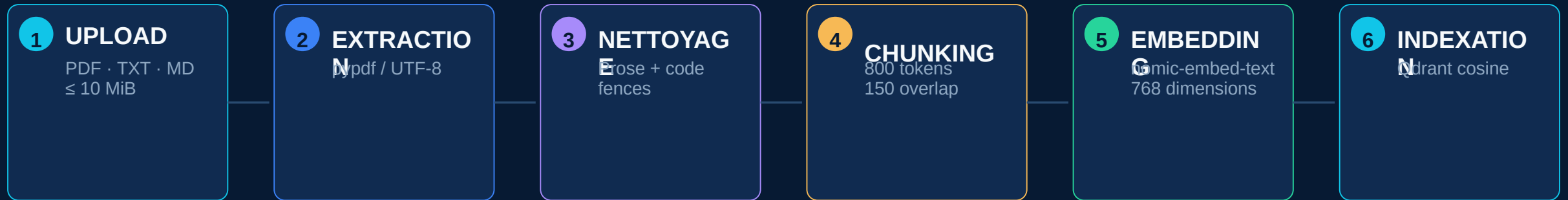
8 services Docker Compose · Ollama sur l'hôte · Gemini externe



Séparation logique forte, sans complexité artificielle de microservices.

Ingestion documentaire asynchrone

La requête HTTP reste rapide ; le traitement lourd est délégué au worker.



CYCLE DE STATUT

queued → **processing** → **indexed**

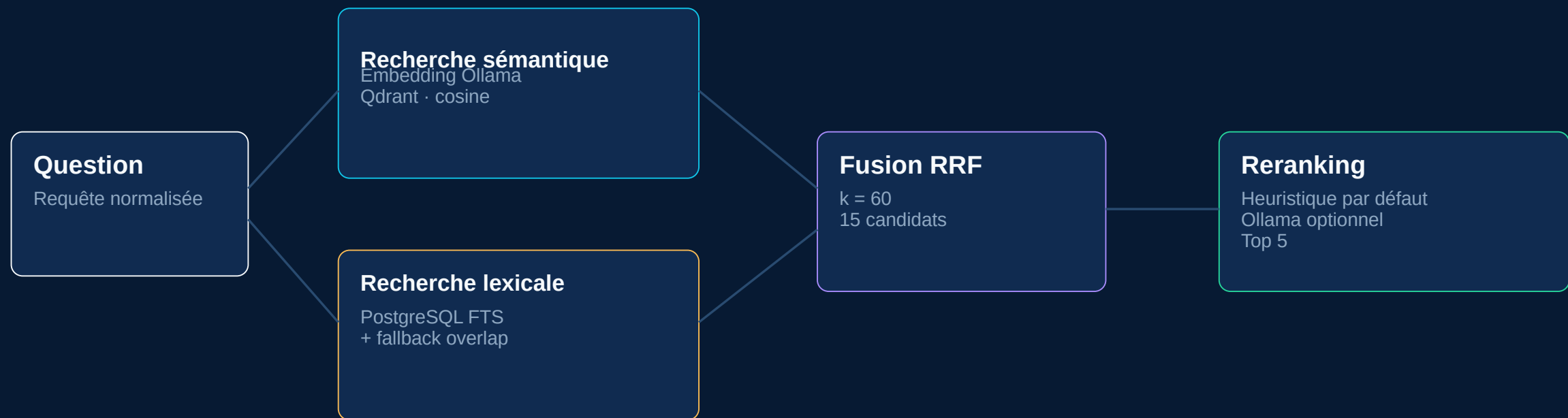
failed
(retry contrôlé)

Celery + Redis

Batch embeddings = 4

Isolation workspace

Recherche hybride : rappel d'abord, précision ensuite



Hybrid = Qdrant sémantique + PostgreSQL lexical · filtrés par workspace · fusionnés par rang.

Pipeline spécialisé en 7 rôles

Les rôles sont orchestrés séquentiellement ; tous ne sont pas des agents LLM autonomes.



Sortie : réponse finale ancrée + citations + évaluation + trace persistée

Streaming et traçabilité : chaque réponse laisse une preuve



L'assistant message devient l'ancre de la lignée d'exécution



Sécurité et gouvernance intégrées

La sécurité est appliquée à la route, à la base relationnelle et à la recherche vectorielle.

IDENTITÉ

JWT signé
Mot de passe bcrypt
Utilisateur actif
Session côté client

AUTORISATION

Rôles user / admin / super-admin
Dépendances FastAPI
Permissions propriétaire

ISOLATION

Workspace membership
Filtre PostgreSQL
Payload filter Qdrant

AUDIT

Actions administratives
Acteur + cible + raison
IP + user-agent

Principe : une requête n'accède jamais à un workspace par simple connaissance de son identifiant.

RAGOps : observer le système logiciel et la qualité RAG

MÉTRIQUES

RUNTIME

Prometheus + Grafana

- débit HTTP
- latence p95
- requêtes RAG
- tokens LLM
- ingestion

ÉVALUATIONS

QUALITÉ

PostgreSQL + traces

- fidélité
- pertinence
- précision contexte
- hallucination
- latence agents

RAGOPS

OPÉRATIONS

Control Tower

- couverture embeddings
- santé workspaces
- cohérence Qdrant
- échecs ingestion
- retry contrôlé

Score santé = 40% couverture + 20% documents + 20% fidélité + 10% faible hallucination + 10% retrieval

Démonstration : de la source à la preuve en 4 minutes

Scénario conseillé : utiliser un document technique court et une question dont la réponse est clairement localisable.

01

PRÉPARER

Choisir un workspace
Uploader un PDF/TXT
Attendre : indexed

02

INTERROGER

Poser une question
Observer les tokens SSE
Ouvrir les citations

03

EXPLIQUER

Ouvrir AI Execution Studio
Montrer retrieval + rerank
Lire les 4 scores

04

AUDITER

Ouvrir Trace Inspector
Montrer agents + chunks
Finir dans RAGOps

Plan B soutenance : garder trois captures locales — Chat cité · Trace Inspector · RAGOps Control Tower.

Résultats : une chaîne fonctionnelle et vérifiable

118

cas de test backend versionnés

39

cas de test frontend versionnés

36

scénarios d'intégration live versionnés

CAPACITÉS DÉMONSTRABLES

Ingestion

fichier → chunks → vecteurs

Retrieval

semantic + keyword + RRF

Réponse

streaming + citations

Qualité

4 scores + correcteur

Trace

agents + chunks + usage

Ops

RAGOps + Prometheus/Grafana

À joindre au dossier final : capture d'un run pytest/Vitest et mesures de la démonstration réelle.

Limites assumées et perspectives

AUJOURD'HUI

Docker Compose mono-hôte

Stockage fichiers local

Évaluation proxy / LLM judge

Reranking heuristique par défaut

Pas de benchmark RAGAS

Pas de tests E2E navigateur automatisés

PROCHAINE ÉTAPE

Kubernetes et stockage objet

Routage multi-LLM dynamique

Cross-encoder de reranking

Jeu d'évaluation + RAGAS

Support multimodal

Alertes et SLO RAG

Une limite clairement mesurée devient une feuille de route, pas une faiblesse cachée.

CONCLUSION

DevPilot AI rend le RAG inspectable

Pas seulement une réponse — une réponse accompagnée de ses preuves.

ANCRÉ

Documents privés
Recherche hybride

AUDITABLE

Citations
Trace complète

OPÉRABLE

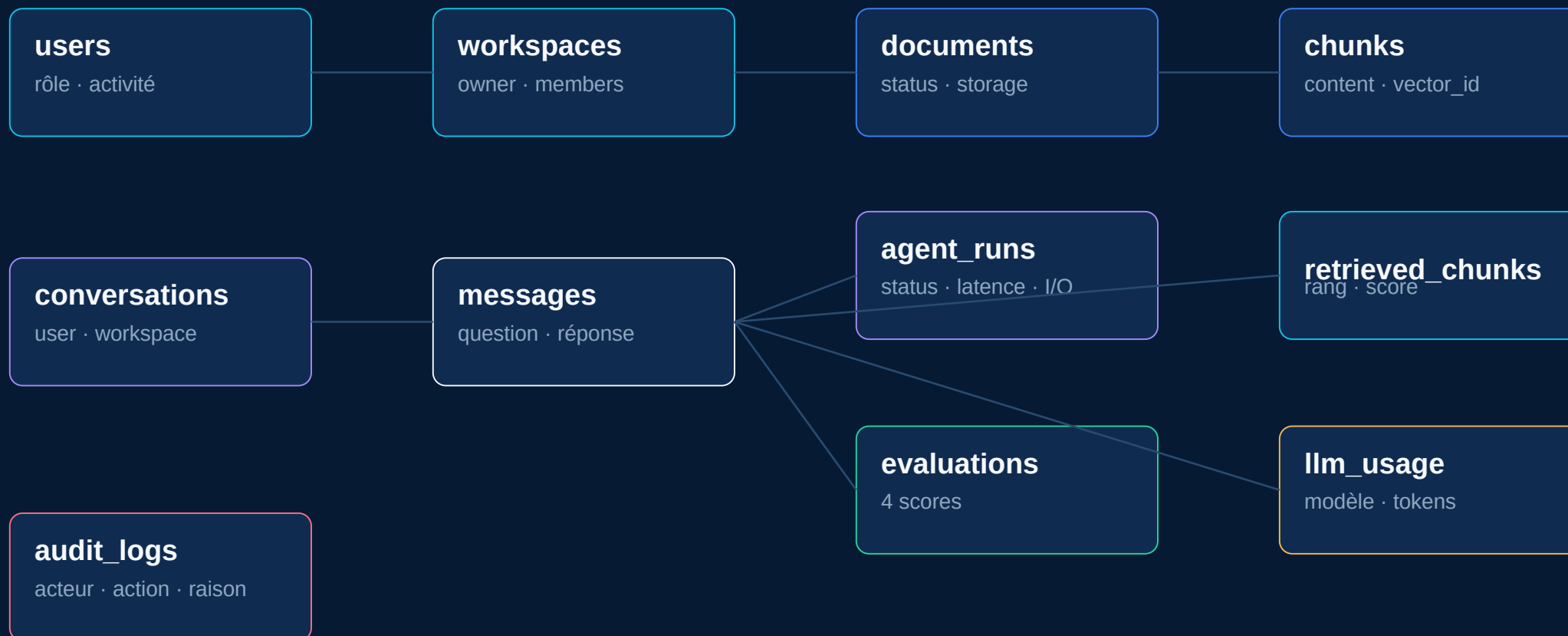
Évaluation
RAGOps

Merci pour votre attention

Questions & discussion

Annexe · Modèle de données de traçabilité

11 tables métier : identité, knowledge base, conversation, preuves et gouvernance.



Annexe · Paramètres réels du pipeline

CHUNK

800

tokens

OVERLAP

150

tokens

EMBEDDING

768

dimensions

CANDIDATS

15

retrieval

CONTEXTE

5

chunks

RRF

60

constante k

PROMPT

6000

caractères max

UPLOAD

10

MiB max

Les valeurs sont configurables ; celles-ci correspondent aux valeurs par défaut du dépôt vérifié.